

z4xaqamuc

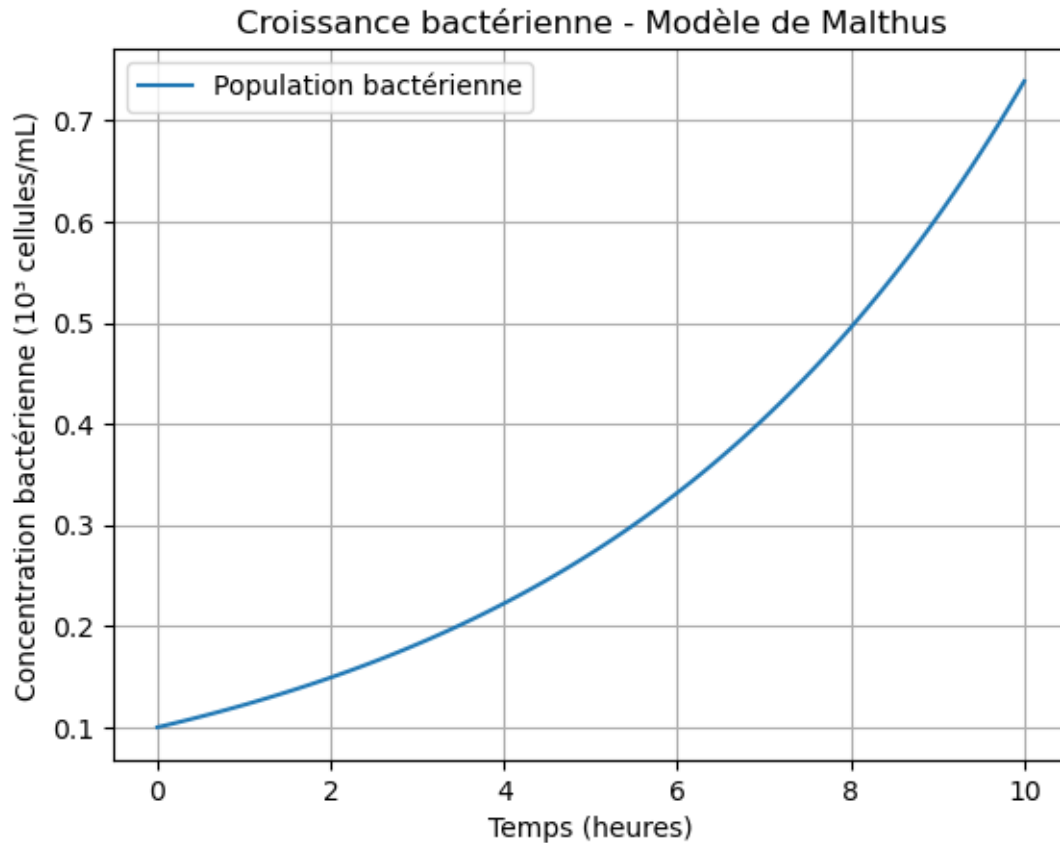
May 26, 2026

```
[2]: import numpy as np
import matplotlib.pyplot as plt

# Paramètres de la simulation
rmax = 0.2 # taux de croissance maximal (h-1)
C0 = 0.1 # Concentration des cellules bactériennes au début de la simulation
      ↪ (103 cellules.mL-1)
T = 10 # durée totale de la simulation (heures)
dt = 0.1 # pas de temps (heures)
N_steps = int(T / dt)

# Simulation
time = np.linspace(0, T, N_steps)
C = C0 * np.exp(rmax * time)

# Tracé du graphique
plt.title('Croissance bactérienne - Modèle de Malthus') # Titre du graphique
plt.plot(time, C, label='Population bactérienne') # Nom de la courbe
plt.xlabel('Temps (heures)') # Axe des X
plt.ylabel('Concentration bactérienne (103 cellules/mL)') # Axe des Y
plt.grid(True)
plt.legend()
plt.show()
```



Type de courbe: c'est une courbe exponentielle Le temps de dedoublement de la population: ce temps est donné par $T_d = \ln(2)/r_{\max} \Rightarrow T_d = \ln(2)/0.2 \Rightarrow T_d = 3.47 \text{ h}$ Donc la population double environ toutes les 3,5 heures

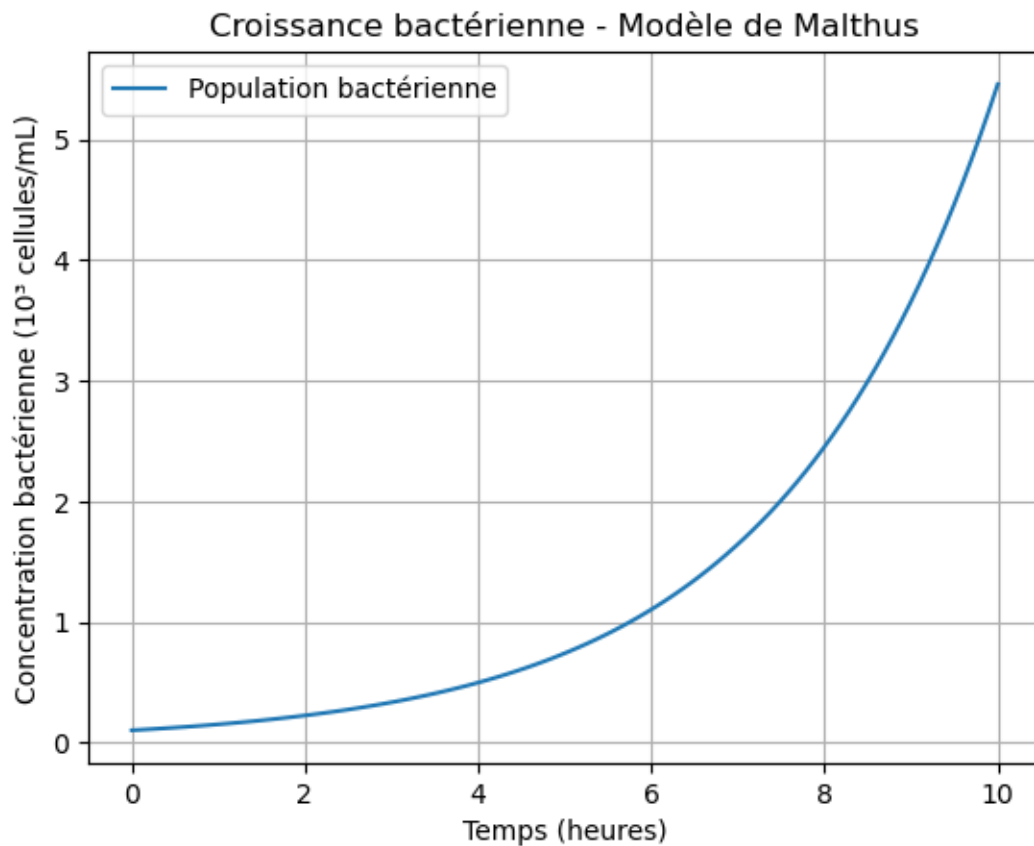
```
[3]: import numpy as np
import matplotlib.pyplot as plt

# Paramètres de la simulation
rmax = 0.4 # taux de croissance maximal (h-1)
C0 = 0.1 # Concentration des cellules bactériennes au début de la simulation
        ↪ (103 cellules.mL-1)
T = 10 # durée totale de la simulation (heures)
dt = 0.1 # pas de temps (heures)
N_steps = int(T / dt)

# Simulation
time = np.linspace(0, T, N_steps)
C = C0 * np.exp(rmax * time)

# Tracé du graphique
```

```
plt.title('Croissance bactérienne - Modèle de Malthus') # Titre du graphique
plt.plot(time, C, label='Population bactérienne') # Nom de la courbe
plt.xlabel('Temps (heures)') # Axe des X
plt.ylabel('Concentration bactérienne (103 cellules/mL)') # Axe des Y
plt.grid(True)
plt.legend()
plt.show()
```



```
[4]: import numpy as np
import matplotlib.pyplot as plt

# Paramètres de la simulation
rmax = 0.1 # taux de croissance maximal (h-1)
C0 = 0.1 # Concentration des cellules bactériennes au début de la simulation
        ↪ (103 cellules.mL-1)
T = 10 # durée totale de la simulation (heures)
dt = 0.1 # pas de temps (heures)
N_steps = int(T / dt)

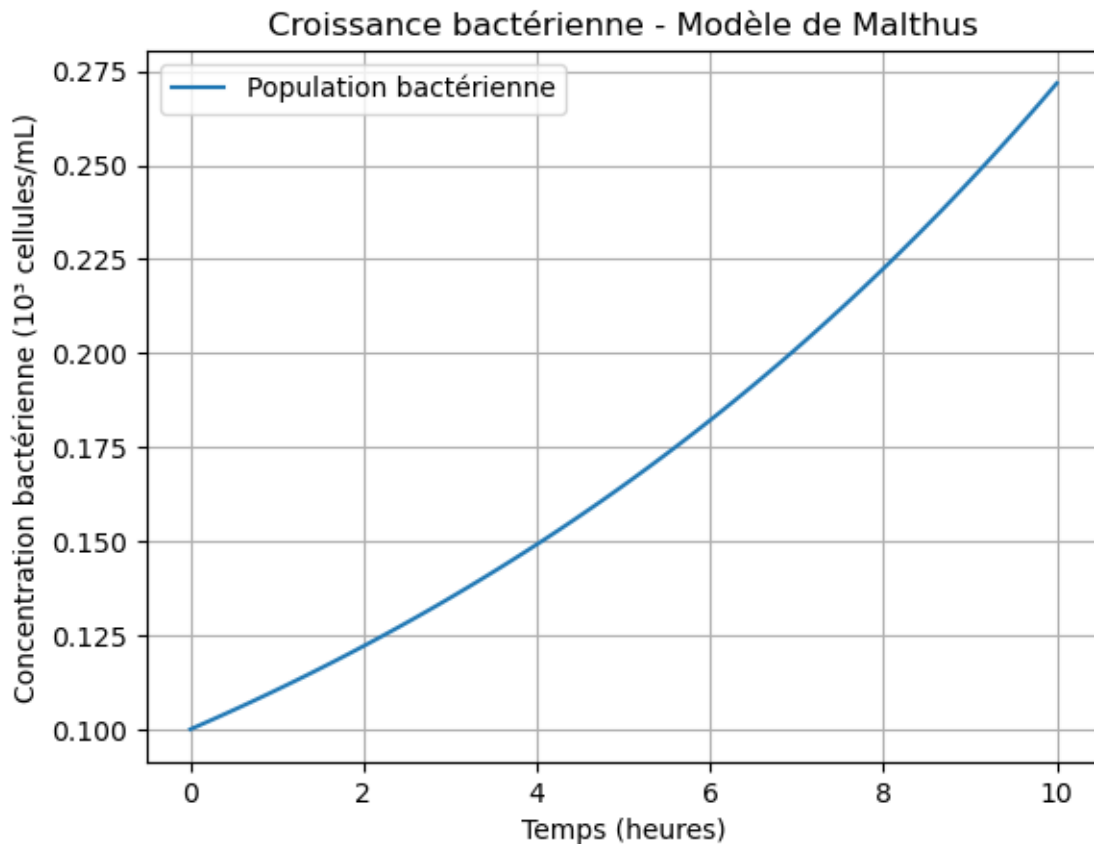
# Simulation
```

```

time = np.linspace(0, T, N_steps)
C = C0 * np.exp(rmax * time)

# Tracé du graphique
plt.title('Croissance bactérienne - Modèle de Malthus') # Titre du graphique
plt.plot(time, C, label='Population bactérienne') # Nom de la courbe
plt.xlabel('Temps (heures)') # Axe des X
plt.ylabel('Concentration bactérienne (103 cellules/mL)') # Axe des Y
plt.grid(True)
plt.legend()
plt.show()

```



```

[5]: import numpy as np
import matplotlib.pyplot as plt

# Paramètres de la simulation
rmax = 0 # taux de croissance maximal (h-1)
C0 = 0.1 # Concentration des cellules bactériennes au début de la simulation
        ↪ (103 cellules.mL-1)
T = 10 # durée totale de la simulation (heures)

```

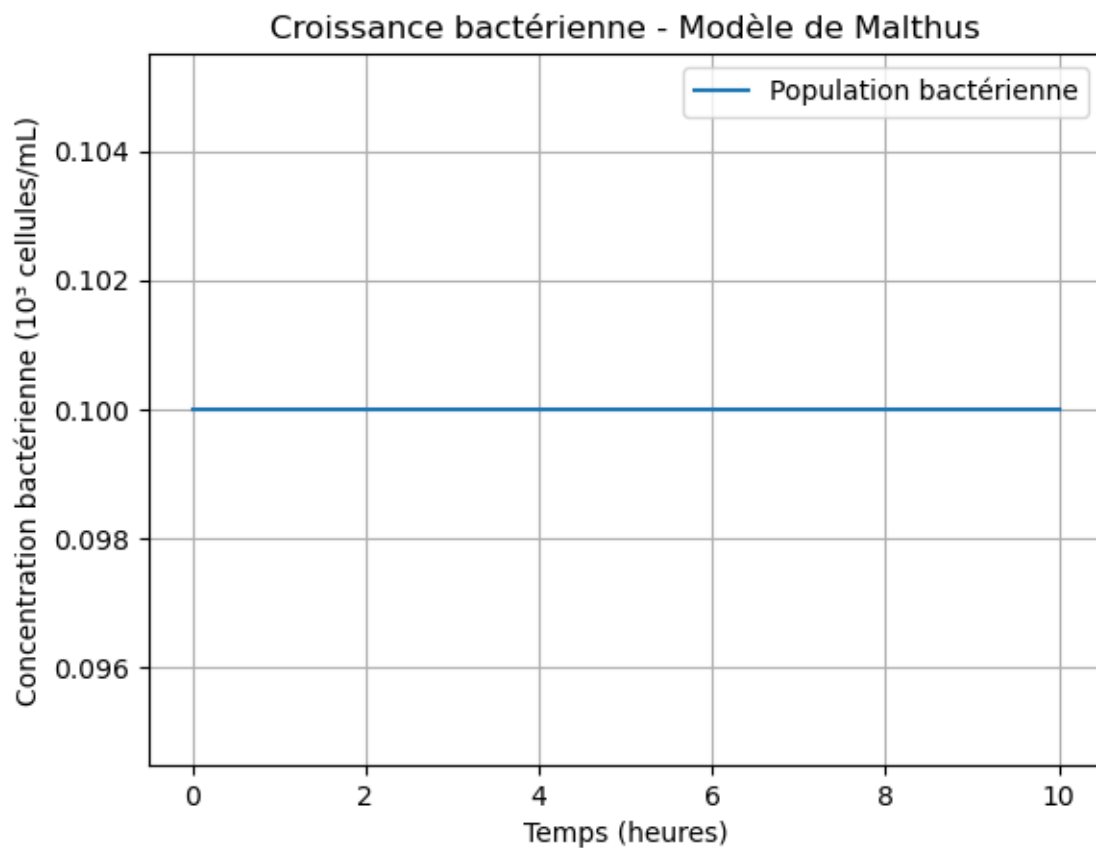
```

dt = 0.1 # pas de temps (heures)
N_steps = int(T / dt)

# Simulation
time = np.linspace(0, T, N_steps)
C = C0 * np.exp(rmax * time)

# Tracé du graphique
plt.title('Croissance bactérienne - Modèle de Malthus') # Titre du graphique
plt.plot(time, C, label='Population bactérienne') # Nom de la courbe
plt.xlabel('Temps (heures)') # Axe des X
plt.ylabel('Concentration bactérienne (103 cellules/mL)') # Axe des Y
plt.grid(True)
plt.legend()
plt.show()

```



```

[6]: import numpy as np
import matplotlib.pyplot as plt

# Paramètres de la simulation

```

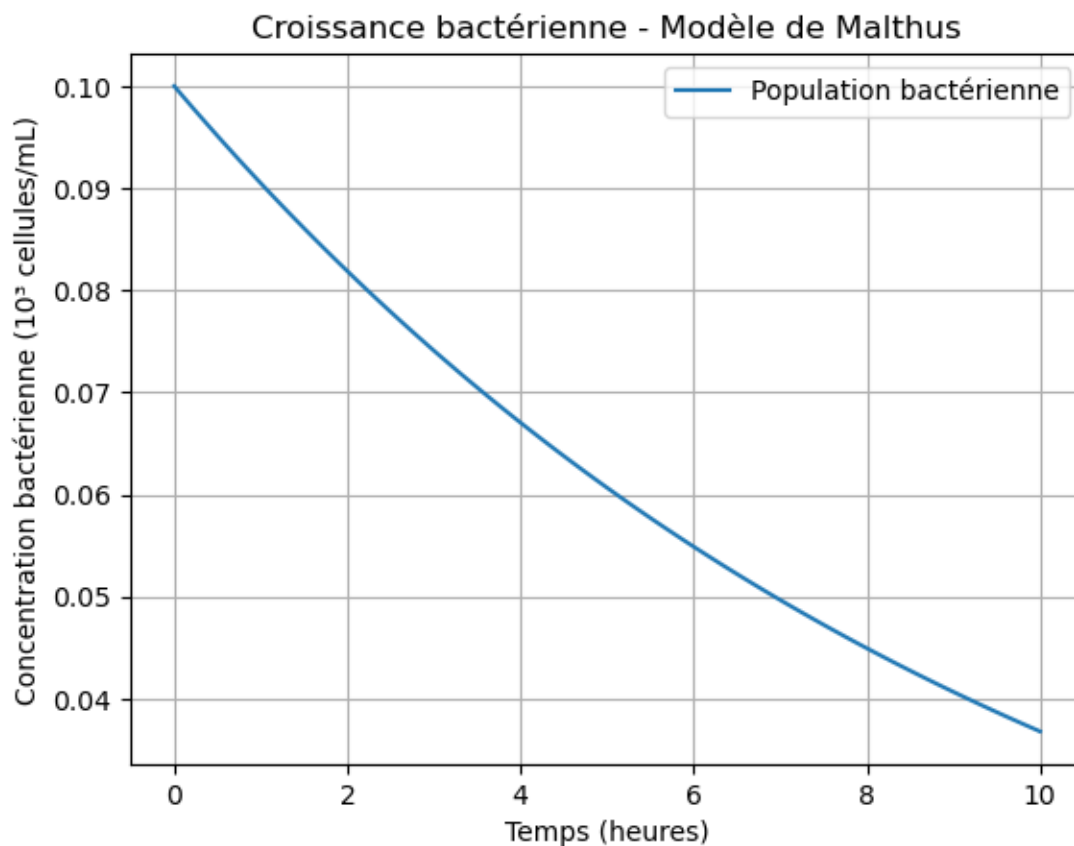
```

rmax = -0.1 # taux de croissance maximal (h-1)
C0 = 0.1 # Concentration des cellules bactériennes au début de la simulation
↳ (103 cellules/mL)
T = 10 # durée totale de la simulation (heures)
dt = 0.1 # pas de temps (heures)
N_steps = int(T / dt)

# Simulation
time = np.linspace(0, T, N_steps)
C = C0 * np.exp(rmax * time)

# Tracé du graphique
plt.title('Croissance bactérienne - Modèle de Malthus') # Titre du graphique
plt.plot(time, C, label='Population bactérienne') # Nom de la courbe
plt.xlabel('Temps (heures)') # Axe des X
plt.ylabel('Concentration bactérienne (103 cellules/mL)') # Axe des Y
plt.grid(True)
plt.legend()
plt.show()

```



Analyse des différentes valeurs de r_{max} à $r_{max}=0.4$ La croissance est beaucoup plus rapide. La pente de la courbe augmente fortement. La population augmente rapidement. à $r_{max}=0.1$ On a une Croissance plus lente. La population augmente progressivement. à $r_{max}=0$ il y'a aucune croissance. La population reste constante. à $r_{max}=-0.1$ La population diminue exponentiellement. Les bactéries meurent plus vite qu'elles ne se divisent. Les limitations de ce model sont:

il ne prend pas en compte le taux de mortalité des cellules, le nombre de division cellulaire par unité de temps, l'effet des antibiotiques Il suppose des ressources infinies ;

Pour le rendre plus réaliste: Il faut justement Une limitation de nutriment; un taux de mortalité; l'effet des antibiotiques; Connaitre le nombre de division cellulaire

```
[7]: import numpy as np
import matplotlib.pyplot as plt

# =====
# PARAMÈTRES DE LA SIMULATION
# =====
T = 300          # Durée totale (heures)
dt = 0.1         # Pas de temps (heures)
N_steps = int(T / dt) # Nombre de pas de temps

# =====
# PARAMÈTRES DE LA SOUCHE
# =====
rmax = 0.10      # Taux de croissance maximal (h-1)
K = 50          # Affinité pour le substrat (g/L)
m0 = 0.07       # Taux de mortalité minimal (h-1)
C0 = 3          # Concentration initiale de bactéries (103 cellules/mL)

# =====
# PARAMÈTRES DU SUBSTRAT (GLUCOSE)
# =====
initial_glucose = 400 # Concentration initiale de glucose (g/L)
glucose_inflow = 0    # Flux entrant de glucose (g/L/h)

# =====
# INITIALISATION DES TABLEAUX
# =====
C = np.zeros(N_steps)      # Concentration bactérienne
glucose_level = np.zeros(N_steps)
growth_rate = np.zeros(N_steps)
mortality_rate = np.zeros(N_steps)

# =====
# CONDITIONS INITIALES
# =====
C[0] = C0
```

```

glucose_level[0] = initial_glucose
growth_rate[0] = rmax * glucose_level[0] / (K + glucose_level[0])
mortality_rate[0] = m0

# =====
# SIMULATION
# =====
for i in range(1, N_steps):
    # Calcul des taux
    r = rmax * glucose_level[i-1] / (K + glucose_level[i-1])

    # Mise à jour de la population bactérienne
    C[i] = C[i-1] + dt * (r - m0) * C[i-1]

    # Mise à jour du glucose (consommation + flux entrant)
    glucose_level[i] = glucose_level[i-1] - dt * r * C[i-1] + glucose_inflow * dt

    # Stockage des taux
    growth_rate[i] = r
    mortality_rate[i] = m0

# =====
# AXE DES X EN HEURES
# =====
time = np.linspace(0, T, N_steps)

# =====
# TRACÉ DES COURBES
# =====
plt.figure(figsize=(12, 8))

# 1. Concentration bactérienne
plt.subplot(4, 1, 1)
plt.plot(time, C, label='Nombre de bactéries', color='blue')
plt.title('Concentration des bactéries - Modèle de Monod')
plt.ylabel('103 cellules/mL')
plt.grid(True)

# 2. Concentration de glucose
plt.subplot(4, 1, 2)
plt.plot(time, glucose_level, label='Concentration de glucose', color='green')
plt.title('Concentration de glucose')
plt.ylabel('g/L')
plt.grid(True)

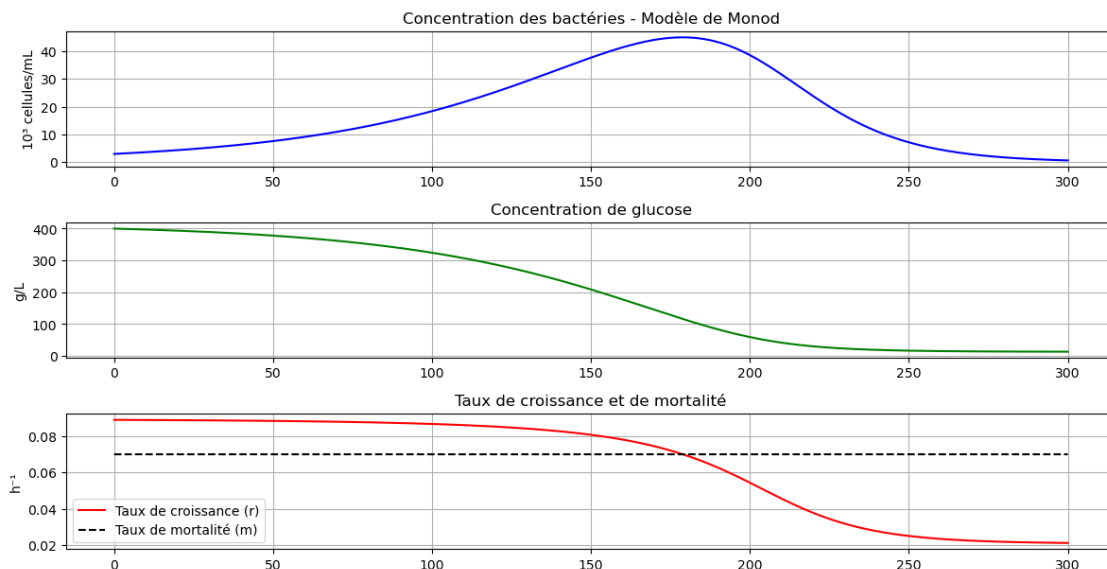
# 3. Taux de croissance ET mortalité (sur le même graphique)

```



```
plt.subplot(4, 1, 3)
plt.plot(time, growth_rate, label='Taux de croissance (r)', color='red')
plt.plot(time, mortality_rate, label='Taux de mortalité (m)', color='black',
         linestyle='--') # Ligne noire en pointillé
plt.title('Taux de croissance et de mortalité')
plt.ylabel('h-1')
plt.legend() # Affiche la légende pour les deux courbes
plt.grid(True)

plt.tight_layout()
plt.show()
```



1) Description des courbes: La population bactérienne augmente au début car le glucose est abondant. Ensuite la croissance ralentit. Puis la population chute lorsque le glucose devient insuffisant. Glucose Le glucose diminue progressivement car il est consommé par les bactéries. Taux de croissance Il diminue avec la baisse du glucose. Taux de mortalité Il reste constant dans cette simulation.

2) La population s'écroule parce que le glucose devient limitant. Le taux de croissance devient inférieur au taux de mortalité alors les bactéries meurent plus vite qu'elles ne se multiplient. 3) le facteur limitant est le glucose 4) La population atteint son maximum lorsque le taux de croissance égal au taux de mortalité 5) Valeur graphique de K: K correspond à la concentration en substrat pour laquelle r_{max} est divisé par 2 $\Rightarrow K = 50g/L$

```
[9]: import numpy as np
import matplotlib.pyplot as plt

# =====
# PARAMÈTRES DE LA SIMULATION
# =====
```

```

T = 300          # Durée totale (heures)
dt = 0.1         # Pas de temps (heures)
N_steps = int(T / dt) # Nombre de pas de temps

# =====
# PARAMÈTRES DE LA SOUCHE
# =====
rmax = 0.20      # Taux de croissance maximal (h-1)
K = 50           # Affinité pour le substrat (g/L)
m0 = 0.07        # Taux de mortalité minimal (h-1)
C0 = 3           # Concentration initiale de bactéries (103 cellules/mL)

# =====
# PARAMÈTRES DU SUBSTRAT (GLUCOSE)
# =====
initial_glucose = 400 # Concentration initiale de glucose (g/L)
glucose_inflow = 0    # Flux entrant de glucose (g/L/h)

# =====
# INITIALISATION DES TABLEAUX
# =====
C = np.zeros(N_steps)      # Concentration bactérienne
glucose_level = np.zeros(N_steps)
growth_rate = np.zeros(N_steps)
mortality_rate = np.zeros(N_steps)

# =====
# CONDITIONS INITIALES
# =====
C[0] = C0
glucose_level[0] = initial_glucose
growth_rate[0] = rmax * glucose_level[0] / (K + glucose_level[0])
mortality_rate[0] = m0

# =====
# SIMULATION
# =====
for i in range(1, N_steps):
    # Calcul des taux
    r = rmax * glucose_level[i-1] / (K + glucose_level[i-1])

    # Mise à jour de la population bactérienne
    C[i] = C[i-1] + dt * (r - m0) * C[i-1]

    # Mise à jour du glucose (consommation + flux entrant)
    glucose_level[i] = glucose_level[i-1] - dt * r * C[i-1] + glucose_inflow *
    ↪dt

```

```

    # Stockage des taux
    growth_rate[i] = r
    mortality_rate[i] = m0

# =====
# AXE DES X EN HEURES
# =====
time = np.linspace(0, T, N_steps)

# =====
# TRACÉ DES COURBES
# =====
plt.figure(figsize=(12, 8))

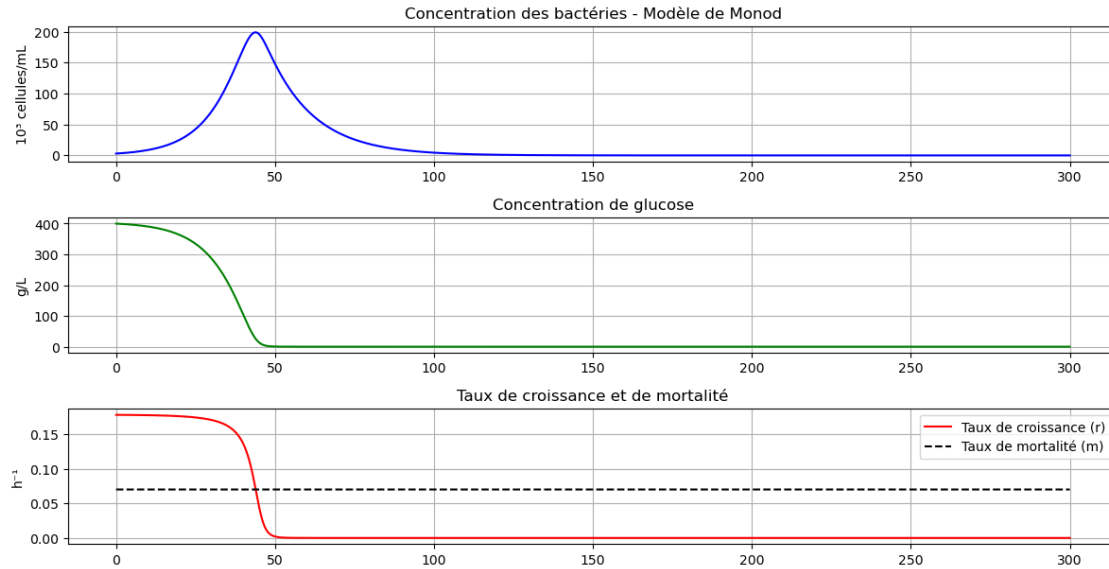
# 1. Concentration bactérienne
plt.subplot(4, 1, 1)
plt.plot(time, C, label='Nombre de bactéries', color='blue')
plt.title('Concentration des bactéries - Modèle de Monod')
plt.ylabel('103 cellules/mL')
plt.grid(True)

# 2. Concentration de glucose
plt.subplot(4, 1, 2)
plt.plot(time, glucose_level, label='Concentration de glucose', color='green')
plt.title('Concentration de glucose')
plt.ylabel('g/L')
plt.grid(True)

# 3. Taux de croissance ET mortalité (sur le même graphique)
plt.subplot(4, 1, 3)
plt.plot(time, growth_rate, label='Taux de croissance (r)', color='red')
plt.plot(time, mortality_rate, label='Taux de mortalité (m)', color='black',
        linestyle='--') # Ligne noire en pointillé
plt.title('Taux de croissance et de mortalité')
plt.ylabel('h-1')
plt.legend() # Affiche la légende pour les deux courbes
plt.grid(True)

plt.tight_layout()
plt.show()

```



```
[10]: import numpy as np
import matplotlib.pyplot as plt

# =====
# PARAMÈTRES DE LA SIMULATION
# =====
T = 300          # Durée totale (heures)
dt = 0.1         # Pas de temps (heures)
N_steps = int(T / dt) # Nombre de pas de temps

# =====
# PARAMÈTRES DE LA SOUCHE
# =====
rmax = 0.20      # Taux de croissance maximal ( $h^{-1}$ )
K = 50           # Affinité pour le substrat (g/L)
m0 = 0.09        # Taux de mortalité minimal ( $h^{-1}$ )
C0 = 3           # Concentration initiale de bactéries ( $10^3$  cellules/mL)

# =====
# PARAMÈTRES DU SUBSTRAT (GLUCOSE)
# =====
initial_glucose = 400 # Concentration initiale de glucose (g/L)
glucose_inflow = 0    # Flux entrant de glucose (g/L/h)

# =====
# INITIALISATION DES TABLEAUX
# =====
C = np.zeros(N_steps) # Concentration bactérienne
```

```

glucose_level = np.zeros(N_steps)
growth_rate = np.zeros(N_steps)
mortality_rate = np.zeros(N_steps)

# =====
# CONDITIONS INITIALES
# =====
C[0] = C0
glucose_level[0] = initial_glucose
growth_rate[0] = rmax * glucose_level[0] / (K + glucose_level[0])
mortality_rate[0] = m0

# =====
# SIMULATION
# =====
for i in range(1, N_steps):
    # Calcul des taux
    r = rmax * glucose_level[i-1] / (K + glucose_level[i-1])

    # Mise à jour de la population bactérienne
    C[i] = C[i-1] + dt * (r - m0) * C[i-1]

    # Mise à jour du glucose (consommation + flux entrant)
    glucose_level[i] = glucose_level[i-1] - dt * r * C[i-1] + glucose_inflow * ↳dt

    # Stockage des taux
    growth_rate[i] = r
    mortality_rate[i] = m0

# =====
# AXE DES X EN HEURES
# =====
time = np.linspace(0, T, N_steps)

# =====
# TRACÉ DES COURBES
# =====
plt.figure(figsize=(12, 8))

# 1. Concentration bactérienne
plt.subplot(4, 1, 1)
plt.plot(time, C, label='Nombre de bactéries', color='blue')
plt.title('Concentration des bactéries - Modèle de Monod')
plt.ylabel('103 cellules/mL')
plt.grid(True)

```

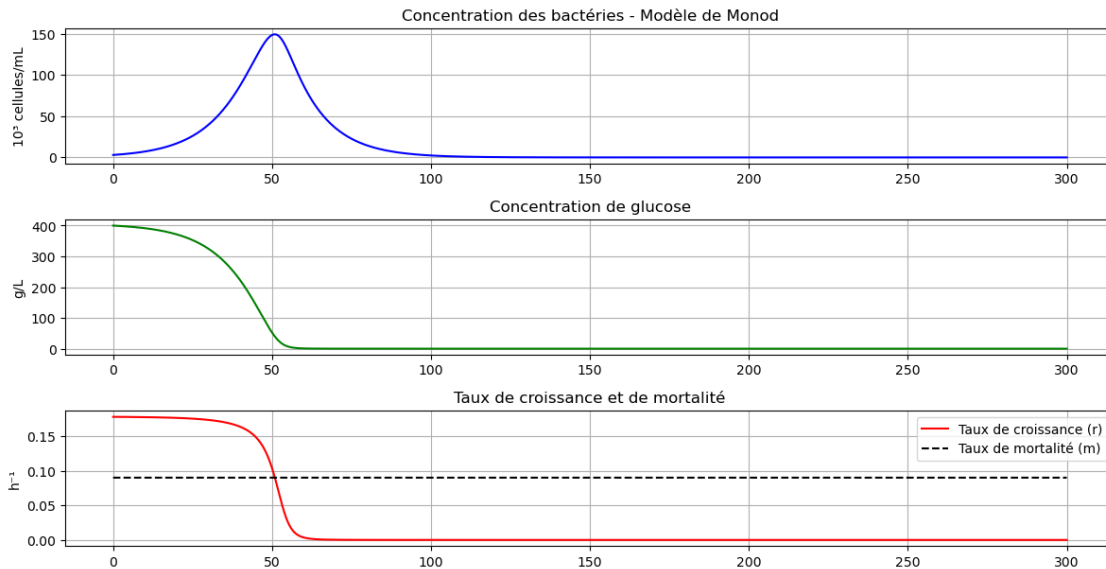
```

# 2. Concentration de glucose
plt.subplot(4, 1, 2)
plt.plot(time, glucose_level, label='Concentration de glucose', color='green')
plt.title('Concentration de glucose')
plt.ylabel('g/L')
plt.grid(True)

# 3. Taux de croissance ET mortalité (sur le même graphique)
plt.subplot(4, 1, 3)
plt.plot(time, growth_rate, label='Taux de croissance (r)', color='red')
plt.plot(time, mortality_rate, label='Taux de mortalité (m)', color='black',
         linestyle='--') # Ligne noire en pointillé
plt.title('Taux de croissance et de mortalité')
plt.ylabel('h-1')
plt.legend() # Affiche la légende pour les deux courbes
plt.grid(True)

plt.tight_layout()
plt.show()

```



```

[11]: import numpy as np
import matplotlib.pyplot as plt

# =====
# PARAMÈTRES DE LA SIMULATION
# =====
T = 300          # Durée totale (heures)
dt = 0.1         # Pas de temps (heures)

```

```

N_steps = int(T / dt) # Nombre de pas de temps

# =====
# PARAMÈTRES DE LA SOUCHE
# =====
rmax = 0.10      # Taux de croissance maximal (h-1)
K = 60           # Affinité pour le substrat (g/L)
m0 = 0.09        # Taux de mortalité minimal (h-1)
C0 = 3 # Concentration initiale de bactéries (103 cellules/mL)

# =====
# PARAMÈTRES DU SUBSTRAT (GLUCOSE)
# =====
initial_glucose = 400 # Concentration initiale de glucose (g/L)
glucose_inflow = 0     # Flux entrant de glucose (g/L/h)

# =====
# INITIALISATION DES TABLEAUX
# =====
C = np.zeros(N_steps)      # Concentration bactérienne
glucose_level = np.zeros(N_steps)
growth_rate = np.zeros(N_steps)
mortality_rate = np.zeros(N_steps)

# =====
# CONDITIONS INITIALES
# =====
C[0] = C0
glucose_level[0] = initial_glucose
growth_rate[0] = rmax * glucose_level[0] / (K + glucose_level[0])
mortality_rate[0] = m0

# =====
# SIMULATION
# =====
for i in range(1, N_steps):
    # Calcul des taux
    r = rmax * glucose_level[i-1] / (K + glucose_level[i-1])

    # Mise à jour de la population bactérienne
    C[i] = C[i-1] + dt * (r - m0) * C[i-1]

    # Mise à jour du glucose (consommation + flux entrant)
    glucose_level[i] = glucose_level[i-1] - dt * r * C[i-1] + glucose_inflow * dt

# Stockage des taux

```

```

    growth_rate[i] = r
    mortality_rate[i] = m0

# =====
# AXE DES X EN HEURES
# =====
time = np.linspace(0, T, N_steps)

# =====
# TRACÉ DES COURBES
# =====
plt.figure(figsize=(12, 8))

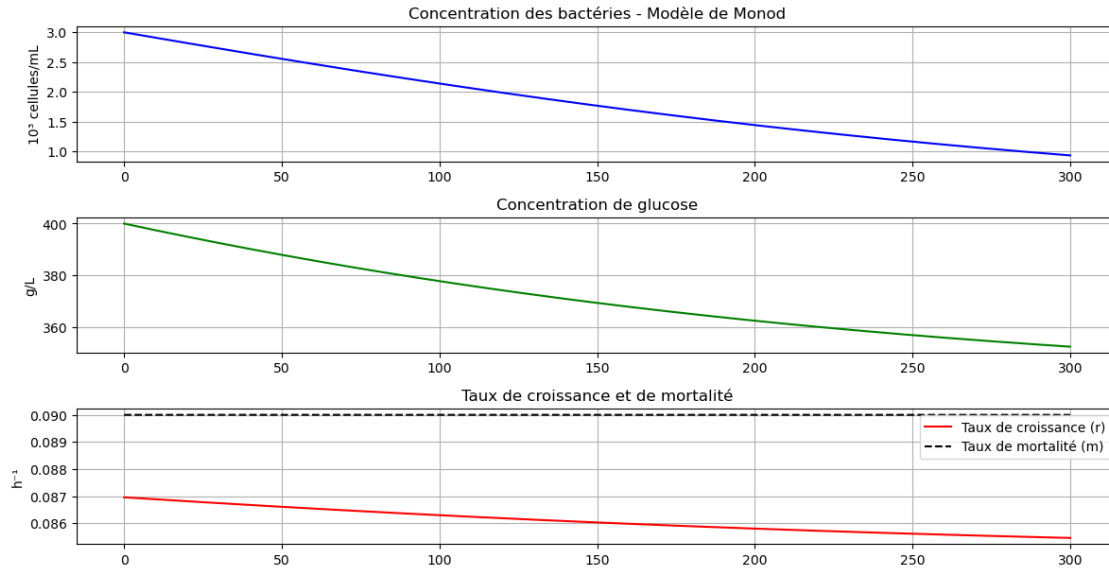
# 1. Concentration bactérienne
plt.subplot(4, 1, 1)
plt.plot(time, C, label='Nombre de bactéries', color='blue')
plt.title('Concentration des bactéries - Modèle de Monod')
plt.ylabel('103 cellules/mL')
plt.grid(True)

# 2. Concentration de glucose
plt.subplot(4, 1, 2)
plt.plot(time, glucose_level, label='Concentration de glucose', color='green')
plt.title('Concentration de glucose')
plt.ylabel('g/L')
plt.grid(True)

# 3. Taux de croissance ET mortalité (sur le même graphique)
plt.subplot(4, 1, 3)
plt.plot(time, growth_rate, label='Taux de croissance (r)', color='red')
plt.plot(time, mortality_rate, label='Taux de mortalité (m)', color='black',
        linestyle='--') # Ligne noire en pointillé
plt.title('Taux de croissance et de mortalité')
plt.ylabel('h-1')
plt.legend() # Affiche la légende pour les deux courbes
plt.grid(True)

plt.tight_layout()
plt.show()

```

```
[12]: import numpy as np
import matplotlib.pyplot as plt

# =====
# PARAMÈTRES DE LA SIMULATION
# =====
T = 300          # Durée totale (heures)
dt = 0.1         # Pas de temps (heures)
N_steps = int(T / dt) # Nombre de pas de temps

# =====
# PARAMÈTRES DE LA SOUCHE
# =====
rmax = 0.10      # Taux de croissance maximal ( $h^{-1}$ )
K = 50           # Affinité pour le substrat (g/L)
m0 = 0.09        # Taux de mortalité minimal ( $h^{-1}$ )
C0 = 3 # Concentration initiale de bactéries ( $10^3$  cellules/mL)

# =====
# PARAMÈTRES DU SUBSTRAT (GLUCOSE)
# =====
initial_glucose = 400 # Concentration initiale de glucose (g/L)
glucose_inflow = 0    # Flux entrant de glucose (g/L/h)

# =====
# INITIALISATION DES TABLEAUX
# =====
C = np.zeros(N_steps) # Concentration bactérienne
```

```

glucose_level = np.zeros(N_steps)
growth_rate = np.zeros(N_steps)
mortality_rate = np.zeros(N_steps)

# =====
# CONDITIONS INITIALES
# =====
C[0] = C0
glucose_level[0] = initial_glucose
growth_rate[0] = rmax * glucose_level[0] / (K + glucose_level[0])
mortality_rate[0] = m0

# =====
# SIMULATION
# =====
for i in range(1, N_steps):
    # Calcul des taux
    r = rmax * glucose_level[i-1] / (K + glucose_level[i-1])

    # Mise à jour de la population bactérienne
    C[i] = C[i-1] + dt * (r - m0) * C[i-1]

    # Mise à jour du glucose (consommation + flux entrant)
    glucose_level[i] = glucose_level[i-1] - dt * r * C[i-1] + glucose_inflow * dt

    # Stockage des taux
    growth_rate[i] = r
    mortality_rate[i] = m0

# =====
# AXE DES X EN HEURES
# =====
time = np.linspace(0, T, N_steps)

# =====
# TRACÉ DES COURBES
# =====
plt.figure(figsize=(12, 8))

# 1. Concentration bactérienne
plt.subplot(4, 1, 1)
plt.plot(time, C, label='Nombre de bactéries', color='blue')
plt.title('Concentration des bactéries - Modèle de Monod')
plt.ylabel('103 cellules/mL')
plt.grid(True)

```

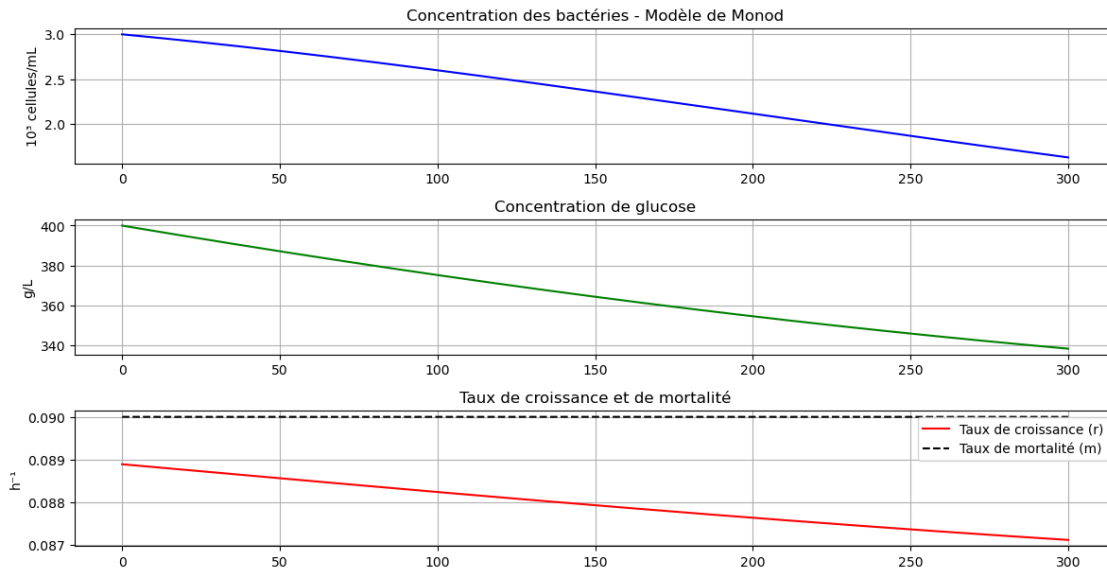
```

# 2. Concentration de glucose
plt.subplot(4, 1, 2)
plt.plot(time, glucose_level, label='Concentration de glucose', color='green')
plt.title('Concentration de glucose')
plt.ylabel('g/L')
plt.grid(True)

# 3. Taux de croissance ET mortalité (sur le même graphique)
plt.subplot(4, 1, 3)
plt.plot(time, growth_rate, label='Taux de croissance (r)', color='red')
plt.plot(time, mortality_rate, label='Taux de mortalité (m)', color='black',
         linestyle='--') # Ligne noire en pointillé
plt.title('Taux de croissance et de mortalité')
plt.ylabel('h-1')
plt.legend() # Affiche la légende pour les deux courbes
plt.grid(True)

plt.tight_layout()
plt.show()

```



- 6) Augmenter r_{max} croissance plus rapide ; consommation plus rapide du glucose. Augmenter m_0 mortalité plus importante ; diminution de la population. Augmenter K affinité plus faible pour le glucose ; croissance plus difficile.
- 7) Il faudrait : un apport continu de glucose ; un équilibre entre croissance et mortalité pour maintenir la population.

```

[13]: import numpy as np
import matplotlib.pyplot as plt

```

```

# =====
# PARAMÈTRES DE LA SIMULATION
# =====
T = 2000          # Durée totale (heures)
dt = 0.1          # Pas de temps (heures)
N_steps = int(T / dt) # Nombre de pas de temps

# =====
# PARAMÈTRES DE LA SOUCHE
# =====
rmax = 0.10       # Taux de croissance maximal (h-1)
K = 50            # Affinité pour le substrat (g/L)
m0 = 0.09         # Taux de mortalité minimal (h-1)
C0 = 10           # Concentration initiale de bactéries (103 cellules/mL)

# =====
# PARAMÈTRES DU SUBSTRAT (GLUCOSE)
# =====
initial_glucose = 140 # Concentration initiale de glucose (g/L)
glucose_inflow = 3    # Flux entrant de glucose (g/L/h)

# =====
# INITIALISATION DES TABLEAUX
# =====
C = np.zeros(N_steps)          # Concentration bactérienne
glucose_level = np.zeros(N_steps)
growth_rate = np.zeros(N_steps)
mortality_rate = np.zeros(N_steps)

# =====
# CONDITIONS INITIALES
# =====
C[0] = C0
glucose_level[0] = initial_glucose
growth_rate[0] = rmax * glucose_level[0] / (K + glucose_level[0])
mortality_rate[0] = m0

# =====
# SIMULATION
# =====
for i in range(1, N_steps):
    # Calcul des taux
    r = rmax * glucose_level[i-1] / (K + glucose_level[i-1])

    # Mise à jour de la population bactérienne
    C[i] = C[i-1] + dt * (r - m0) * C[i-1]

```

```

# Mise à jour du glucose (consommation + flux entrant)
glucose_level[i] = glucose_level[i-1] - dt * r * C[i-1] + glucose_inflow * dt

# Stockage des taux
growth_rate[i] = r
mortality_rate[i] = m0

# =====
# AXE DES X EN HEURES
# =====
time = np.linspace(0, T, N_steps)

# =====
# TRACÉ DES COURBES
# =====
plt.figure(figsize=(12, 8))

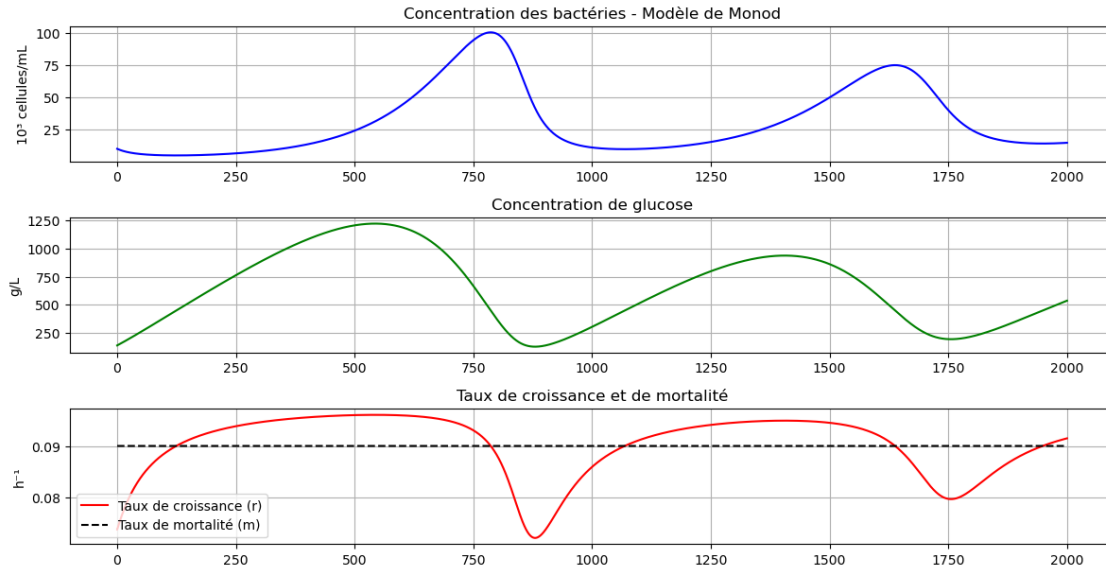
# 1. Concentration bactérienne
plt.subplot(4, 1, 1)
plt.plot(time, C, label='Nombre de bactéries', color='blue')
plt.title('Concentration des bactéries - Modèle de Monod')
plt.ylabel('103 cellules/mL')
plt.grid(True)

# 2. Concentration de glucose
plt.subplot(4, 1, 2)
plt.plot(time, glucose_level, label='Concentration de glucose', color='green')
plt.title('Concentration de glucose')
plt.ylabel('g/L')
plt.grid(True)

# 3. Taux de croissance ET mortalité (sur le même graphique)
plt.subplot(4, 1, 3)
plt.plot(time, growth_rate, label='Taux de croissance (r)', color='red')
plt.plot(time, mortality_rate, label='Taux de mortalité (m)', color='black',
        linestyle='--') # Ligne noire en pointillé
plt.title('Taux de croissance et de mortalité')
plt.ylabel('h-1')
plt.legend() # Affiche la légende pour les deux courbes
plt.grid(True)

plt.tight_layout()
plt.show()

```



Description des courbes La population augmente fortement. Des oscillations apparaissent. Puis un équilibre dynamique se met en place.

On observe des fluctuations parce que les bactéries consomment rapidement le glucose ; lorsque le glucose diminue, la croissance ralentit ; le glucose revient grâce au flux entrant ; la population repart ensuite à la hausse.

Cela crée des oscillations avant stabilisation.

```
[14]: import numpy as np
import matplotlib.pyplot as plt

# =====
# Paramètres pour la souche 1
# =====
rmax_1 = 0.15      # Taux de croissance maximal (souche 1)
m0_1 = 0.06        # Taux de mortalité de base (souche 1)
K_1 = 800          # Constante de demi-saturation (souche 1)

# =====
# Paramètres pour la souche 2
# =====
rmax_2 = 0.15      # Taux de croissance maximal (souche 2)
m0_2 = 0.06        # Taux de mortalité de base (souche 2)
K_2 = 650          # Constante de demi-saturation (souche 2) - affinité plus élevée
                    # pour le glucose

# =====
# Paramètres du glucose
```

```

# =====
initial_glucose = 1200    # Quantité initiale de glucose
glucose_inflow = 4        # Flux entrant de glucose (0 = système fermé)

# =====
# Conditions initiales
# =====
C0_1 = 10    # Concentration initiale de la souche 1
C0_2 = 10    # Concentration initiale de la souche 2

# =====
# Paramètres de la simulation
# =====
dt = 0.1      # Pas de temps
T = 2000      # Durée totale de la simulation
N_steps = int(T / dt) # Nombre de pas de temps

# =====
# Initialisation des tableaux
# =====
C1 = np.zeros(N_steps)
C2 = np.zeros(N_steps)
glucose_level = np.zeros(N_steps)
growth_rate_1 = np.zeros(N_steps)
growth_rate_2 = np.zeros(N_steps)
mortality_rate_1 = np.zeros(N_steps)
mortality_rate_2 = np.zeros(N_steps)

# Conditions initiales
C1[0] = C0_1
C2[0] = C0_2
glucose_level[0] = initial_glucose
growth_rate_1[0] = rmax_1 * glucose_level[0] / (K_1 + glucose_level[0])
growth_rate_2[0] = rmax_2 * glucose_level[0] / (K_2 + glucose_level[0])
mortality_rate_1[0] = m0_1
mortality_rate_2[0] = m0_2

# =====
# Simulation
# =====
for i in range(1, N_steps):
    # Calcul des taux de croissance pour chaque souche (modèle de Monod)
    r_1 = rmax_1 * glucose_level[i-1] / (K_1 + glucose_level[i-1])
    r_2 = rmax_2 * glucose_level[i-1] / (K_2 + glucose_level[i-1])

    # Mortalité variable (augmente si le glucose est faible)
    #m_1 = m0_1 + 0.01 * (1 - glucose_level[i-1] / initial_glucose)

```

```

#m_2 = m0_2 + 0.01 * (1 - glucose_level[i-1] / initial_glucose)

# Mortalité constante
m_1 = m0_1
m_2 = m0_2

# Mise à jour des populations bactériennes
C1[i] = C1[i-1] + dt * (r_1 - m_1) * C1[i-1]
C2[i] = C2[i-1] + dt * (r_2 - m_2) * C2[i-1]

# Consommation totale de glucose (somme des consommations des deux souches)
glucose_consumption = (r_1 * C1[i-1] + r_2 * C2[i-1])
glucose_level[i] = glucose_level[i-1] - dt * glucose_consumption +
↳glucose_inflow * dt

# Stockage des taux
growth_rate_1[i] = r_1
growth_rate_2[i] = r_2
mortality_rate_1[i] = m_1
mortality_rate_2[i] = m_2

# =====
# Temps pour l'axe des abscisses
# =====
time = np.linspace(0, T, N_steps)

# =====
# Tracé des courbes
# =====
plt.figure(figsize=(12, 12))

# Population des bactéries
plt.subplot(4, 1, 1)
plt.plot(time, C1, label='Souche 1', color='blue')
plt.plot(time, C2, label='Souche 2', color='orange')
plt.title('Compétition entre 2 souches - Modèle de Monod')
plt.ylabel('103 cellules/mL')
plt.legend()
plt.grid(True)

# Niveau de glucose
plt.subplot(4, 1, 2)
plt.plot(time, glucose_level, label='Glucose', color='green')
plt.title('Concentration de glucose')
plt.ylabel('g/L')
plt.grid(True)

```



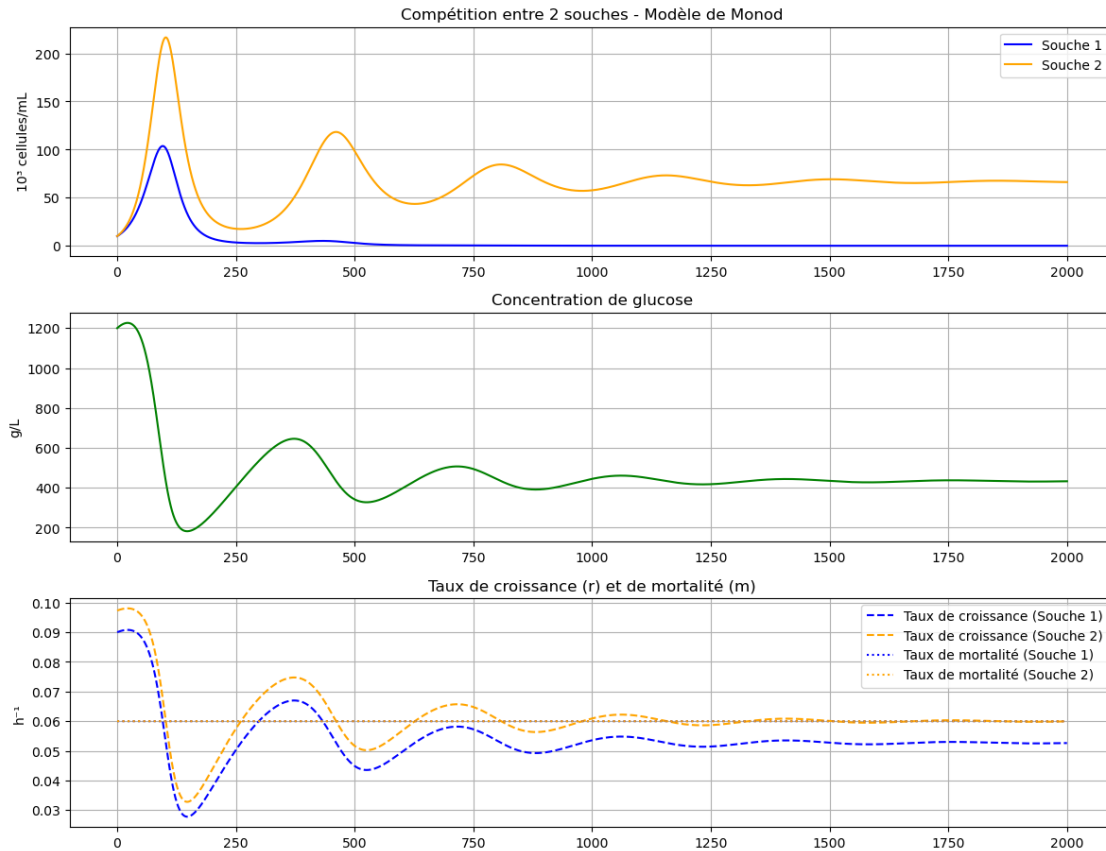
```

# Taux de croissance et mortalité
plt.subplot(4, 1, 3)
plt.plot(time, growth_rate_1, label='Taux de croissance (Souche 1)',
         color='blue', linestyle='--')
plt.plot(time, growth_rate_2, label='Taux de croissance (Souche 2)',
         color='orange', linestyle='--')
plt.plot(time, mortality_rate_1, label='Taux de mortalité (Souche 1)',
         color='blue', linestyle=':')
plt.plot(time, mortality_rate_2, label='Taux de mortalité (Souche 2)',
         color='orange', linestyle=':')
plt.title('Taux de croissance (r) et de mortalité (m)')
plt.ylabel('h')
plt.legend()
plt.grid(True)

# Taux de mortalité
#plt.subplot(4, 1, 4)
#plt.plot(time, mortality_rate_1, label='Taux de mortalité (Souche 1)',
#         color='blue', linestyle=':')
#plt.plot(time, mortality_rate_2, label='Taux de mortalité (Souche 2)',
#         color='orange', linestyle=':')
#plt.title('Taux de mortalité (m) au fil du temps')
#plt.xlabel('Temps')
#plt.ylabel('Taux de mortalité (m)')
#plt.legend()
#plt.grid(True)

plt.tight_layout()
plt.show()

```



[]: 8) Concurrence entre 2 souches:

Description des courbes:

Les deux souches croissent au départ.

La souche 2 devient progressivement dominante.

La souche 1 finit par disparaître.

9) Résultat de la concurrence

La souche 2 possède : un K plus faible ;

donc une meilleure affinité pour le glucose.

Elle exploite mieux les ressources et élimine progressivement la souche 1.

10) Valeur de m1: La souche 1 peut gagner si $m_1 < m_2$

```
[15]: import numpy as np
import matplotlib.pyplot as plt

# =====
# PARAMÈTRES POUR LES SOUCHES
# =====
# Souche 1
```

```

rmax_1 = 0.15      # Taux de croissance maximal (h-1)
m0_1 = 0.06        # Taux de mortalité de base (h-1)
K_1 = 800           # Constante de demi-saturation (g/L)
resistance_1 = 0    # Résistance à l'antibiotique (0 = aucune, 1 = totale)

# Souche 2
rmax_2 = 0.15      # Taux de croissance maximal (h-1)
m0_2 = 0.06        # Taux de mortalité de base (h-1)
K_2 = 650           # Constante de demi-saturation (g/L)
resistance_2 = 0    # Résistance à l'antibiotique

# =====
# PARAMÈTRES DU GLUCOSE
# =====
initial_glucose = 1200    # Concentration initiale de glucose (g/L)
glucose_inflow = 4        # Flux entrant de glucose (g/L/h)

# =====
# PARAMÈTRES DE L'ANTIBIOTIQUE
# =====
initial_antibiotic = 00    # Concentration initiale d'antibiotique (mg/L)
antibiotic_intensity = 3   # Intensité (0 = aucun effet, 1 = mortalité doublée)
antibiotic_decay = 0.001   # Taux de décroissance naturelle (h-1)
antibiotic_inflow = 0      # Flux entrant d'antibiotique (mg/L/h)
antibiotic_start_time = 50 # Temps (en heures) avant l'injection

# =====
# CONDITIONS INITIALES
# =====
CO_1 = 10    # Concentration initiale de la souche 1 (103 cellules/mL)
CO_2 = 10    # Concentration initiale de la souche 2 (103 cellules/mL)

# =====
# PARAMÈTRES DE LA SIMULATION
# =====
dt = 0.1      # Pas de temps (h)
T = 2000      # Durée totale (h)
N_steps = int(T / dt)

# =====
# INITIALISATION DES TABLEAUX
# =====
C1 = np.zeros(N_steps)
C2 = np.zeros(N_steps)
glucose_level = np.zeros(N_steps)
antibiotic_level = np.zeros(N_steps)
growth_rate_1 = np.zeros(N_steps)

```

```

growth_rate_2 = np.zeros(N_steps)
mortality_rate_1 = np.zeros(N_steps)
mortality_rate_2 = np.zeros(N_steps)

# Conditions initiales
C1[0] = C0_1
C2[0] = C0_2
glucose_level[0] = initial_glucose
antibiotic_level[0] = 0
growth_rate_1[0] = rmax_1 * glucose_level[0] / (K_1 + glucose_level[0])
growth_rate_2[0] = rmax_2 * glucose_level[0] / (K_2 + glucose_level[0])
mortality_rate_1[0] = m0_1
mortality_rate_2[0] = m0_2

# =====
# SIMULATION
# =====
for i in range(1, N_steps):
    current_time = i * dt

    # Calcul des taux de croissance
    r_1 = rmax_1 * glucose_level[i-1] / (K_1 + glucose_level[i-1])
    r_2 = rmax_2 * glucose_level[i-1] / (K_2 + glucose_level[i-1])

    # Mise à jour de l'antibiotique (CORRIGÉ)
    if current_time < antibiotic_start_time:
        antibiotic_level[i] = 0
    else:
        if antibiotic_level[i-1] == 0: # Premier pas après l'injection
            antibiotic_level[i] = initial_antibiotic
        else:
            antibiotic_level[i] = antibiotic_level[i-1] * (1 - antibiotic_decay_
↪ dt) + antibiotic_inflow * dt

    # Effet de l'antibiotique sur la mortalité
    if antibiotic_level[i] > 0:
        antibiotic_effect_1 = antibiotic_intensity * (antibiotic_level[i] /_
↪ initial_antibiotic) * (1 - resistance_1)
        antibiotic_effect_2 = antibiotic_intensity * (antibiotic_level[i] /_
↪ initial_antibiotic) * (1 - resistance_2)
    else:
        antibiotic_effect_1 = 0
        antibiotic_effect_2 = 0

    m_1 = m0_1 * (1 + antibiotic_effect_1)
    m_2 = m0_2 * (1 + antibiotic_effect_2)

```

```

# Mise à jour des populations
C1[i] = C1[i-1] + dt * (r_1 - m_1) * C1[i-1]
C2[i] = C2[i-1] + dt * (r_2 - m_2) * C2[i-1]

# Consommation de glucose
glucose_consumption = (r_1 * C1[i-1] + r_2 * C2[i-1])
glucose_level[i] = glucose_level[i-1] - dt * glucose_consumption +
↳ glucose_inflow * dt

# Stockage des taux
growth_rate_1[i] = r_1
growth_rate_2[i] = r_2
mortality_rate_1[i] = m_1
mortality_rate_2[i] = m_2

# =====
# TEMPS POUR L'AXE DES ABCISSES
# =====
time = np.linspace(0, T, N_steps)

# =====
# CALCUL DES ÉCHELLES COMMUNES POUR LES TAUX
# =====
# Trouver le maximum global pour les taux de croissance et mortalité
max_growth = max(np.max(growth_rate_1), np.max(growth_rate_2))
max_mortality = max(np.max(mortality_rate_1), np.max(mortality_rate_2))
y_max_rates = max(max_growth, max_mortality) * 1.1 # +10% de marge

# =====
# TRACÉ DES COURBES
# =====
plt.figure(figsize=(12, 15))

# 1. Populations bactériennes
plt.subplot(5, 1, 1)
plt.plot(time, C1, label=f'Souche 1 (Résistance = {resistance_1})',
↳ color='blue')
plt.plot(time, C2, label=f'Souche 2 (Résistance = {resistance_2})',
↳ color='orange')
plt.axvline(x=antibiotic_start_time, color='red', linestyle='--',
↳ label=f'Injection antibiotique (t={antibiotic_start_time}h)')
plt.title('Compétition entre 2 souches avec antibiotique injecté à t=500h')
plt.ylabel('103 cellules/mL')
plt.legend()
plt.grid(True)
plt.ylim(bottom=0)

```

```

# 2. Concentration de glucose
plt.subplot(5, 1, 2)
plt.plot(time, glucose_level, label='Glucose', color='green')
plt.title('Concentration de glucose')
plt.ylabel('g/L')
plt.legend()
plt.grid(True)
plt.ylim(bottom=0)

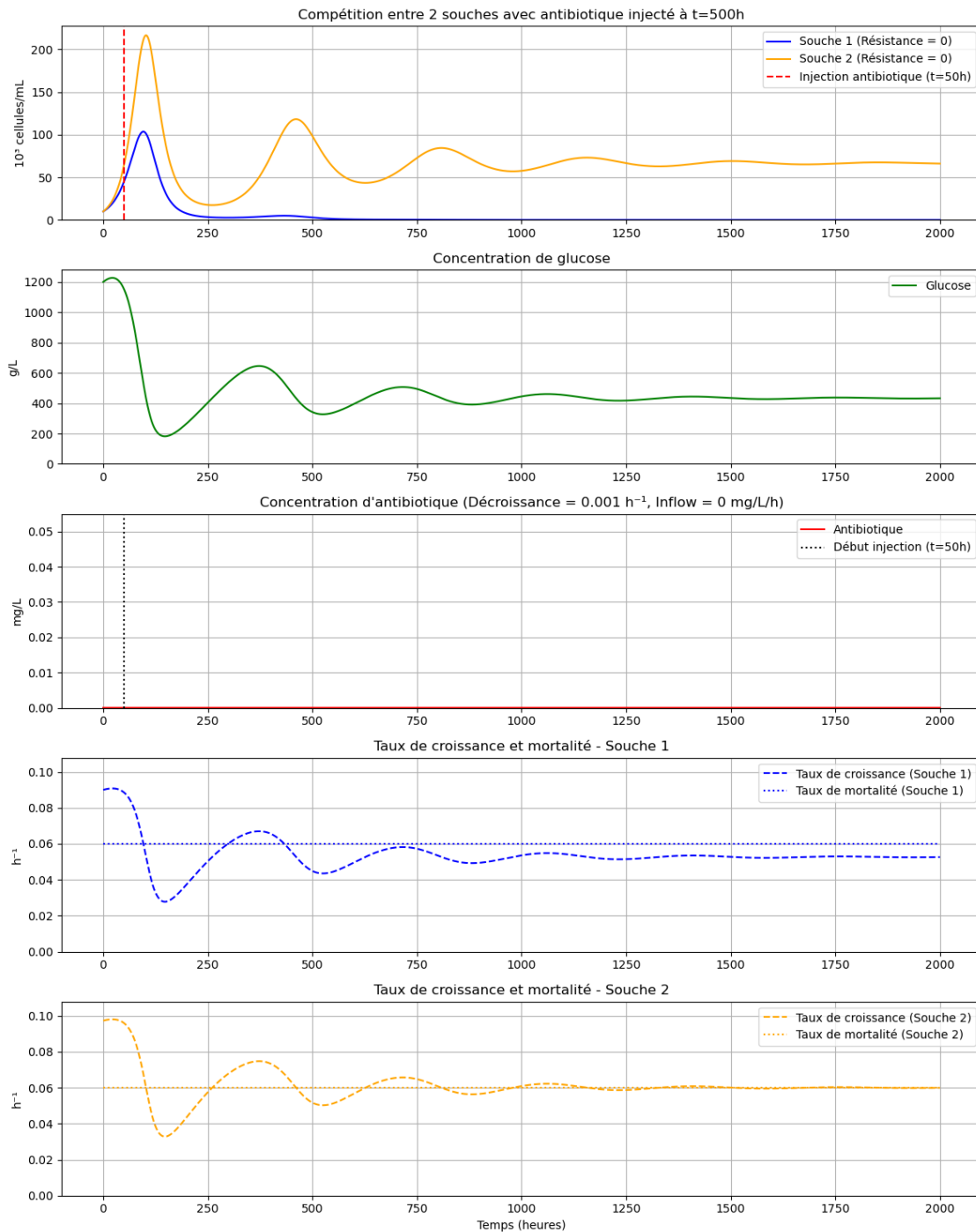
# 3. Concentration d'antibiotique
plt.subplot(5, 1, 3)
plt.plot(time, antibiotic_level, label='Antibiotique', color='red')
plt.axvline(x=antibiotic_start_time, color='black', linestyle=':',
    ↪label=f'Début injection (t={antibiotic_start_time}h)')
plt.title(f'Concentration d\'antibiotique (Décroissance = {antibiotic_decay}
    ↪h-1, Inflow = {antibiotic_inflow} mg/L/h)')
plt.ylabel('mg/L')
plt.legend()
plt.grid(True)
plt.ylim(bottom=0)

# 4. Taux de croissance et mortalité (Souche 1)
plt.subplot(5, 1, 4)
plt.plot(time, growth_rate_1, label='Taux de croissance (Souche 1)',
    ↪color='blue', linestyle='--')
plt.plot(time, mortality_rate_1, label='Taux de mortalité (Souche 1)',
    ↪color='blue', linestyle=':')
plt.title('Taux de croissance et mortalité - Souche 1')
plt.ylabel('h-1')
plt.legend()
plt.grid(True)
plt.ylim(bottom=0, top=y_max_rates) # Échelle commune

# 5. Taux de croissance et mortalité (Souche 2)
plt.subplot(5, 1, 5)
plt.plot(time, growth_rate_2, label='Taux de croissance (Souche 2)',
    ↪color='orange', linestyle='--')
plt.plot(time, mortality_rate_2, label='Taux de mortalité (Souche 2)',
    ↪color='orange', linestyle=':')
plt.title('Taux de croissance et mortalité - Souche 2')
plt.xlabel('Temps (heures)')
plt.ylabel('h-1')
plt.legend()
plt.grid(True)
plt.ylim(bottom=0, top=y_max_rates) # Même échelle que le graphique 4

```

```
plt.tight_layout()
plt.show()
```



Sélection par les antibiotiques Avec antibiotique : Après l'injection, la mortalité augmente fortement. Les deux populations chutent. Puis elles remontent progressivement lorsque l'antibiotique

se dégrade.

Souche totalement résistante (resistance1=1): La souche 1 n'est pas affectée par l'antibiotique. La souche 2 chute fortement. La souche 1 devient dominante.

Souche faiblement résistante (resistance1=0.2)

La souche 1 subit un faible effet antibiotique. Elle survit mieux que la souche 2. Elle devient progressivement majoritaire.

Cela illustre la sélection naturelle de la résistance.

Pollution diffuse d'antibiotique

L'antibiotique reste présent durablement. Les bactéries sensibles sont continuellement désavantagées. Les souches résistantes sont favorisées à long terme.

Cela permet :

les rejets d'antibiotiques ; la sélection de bactéries résistantes dans l'environnement.

PARTIE II: MODEL DE HARDY-WEINBERG – Sélection et dérive génétique

```
[17]: import numpy as np
import matplotlib.pyplot as plt

# Simulation
TAILLE_POP = 1000
GENERATIONS = 50

# Fréquence initiale de l'allèle A
freq_A_initiale = 0.1

# Valeurs sélectives (fitness)
# Cas 1 : sélection (ex : antibiotique ou stress)
fitness_selection = {"AA": 1.0,
                     "Aa": 0.9,
                     "aa": 0.7}

# Cas 2 : pas de sélection (équilibre de Hardy-Weinberg)
fitness_neutre = {"AA": 1.0,
                  "Aa": 1.0,
                  "aa": 1.0}

# Génération d'une population
def generer_population(freq_A, taille):
    freq_a = 1 - freq_A
    probabilites = [
        freq_A ** 2,      # AA
        2 * freq_A * freq_a, # Aa
        freq_a ** 2      # aa
    ]
    return np.random.choice(["AA", "Aa", "aa"], size=taille, p=probabilites)
```



```

# Calcul de la fréquence allélique
def calcul_freq_A(population):
    nb_A = sum(genotype.count("A") for genotype in population)
    return nb_A / (2 * len(population))

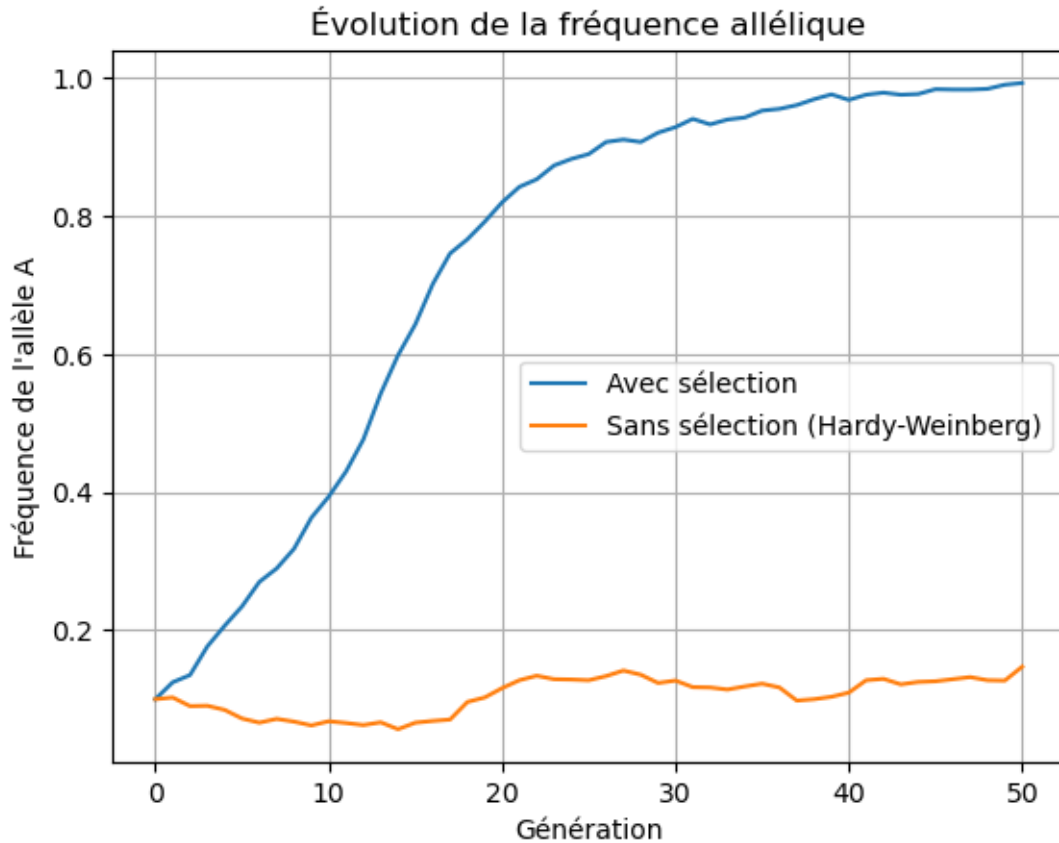
# Application de la sélection
def appliquer_selection(population, fitness):
    poids = [fitness[g] for g in population]
    total = sum(poids)
    probabilites = [p / total for p in poids]
    return np.random.choice(population, size=len(population), p=probabilites)

# Simulation de l'évolution
def simuler(fitness):
    freq_A = freq_A_initiale
    frequences = [freq_A] # Initialisation de la liste
    population = generer_population(freq_A, TAILLE_POP)
    for i in range(GENERATIONS):
        population = appliquer_selection(population, fitness)
        freq_A = calcul_freq_A(population)
        population = generer_population(freq_A, TAILLE_POP)
        frequences.append(freq_A)
    return frequences

# Lancement des simulations
freq_selection = simuler(fitness_selection)
freq_neutre = simuler(fitness_neutre)

# Tracé des courbes
plt.figure()
plt.plot(freq_selection, label="Avec sélection")
plt.plot(freq_neutre, label="Sans sélection (Hardy-Weinberg)")
plt.xlabel("Génération")
plt.ylabel("Fréquence de l'allèle A")
plt.title("Évolution de la fréquence allélique")
plt.legend()
plt.grid(True)
plt.show()

```



Le principe de Hardy-Weinberg :

en absence de sélection, mutation, migration et dérive, les fréquences alléliques restent constantes. Avec sélection Les génotypes les moins adaptés disparaissent progressivement. La fréquence de l'allèle avantageux augmente au fil des générations. Sans sélection (équilibre de Hardy-Weinberg) Les fréquences restent globalement stables. Les petites variations observées proviennent de la dérive génétique aléatoire. Effet de la dérive génétique

La dérive génétique :

est plus forte dans les petites populations ; peut entraîner la disparition aléatoire d'un allèle.